

Soitron Kubernetes Workshop

Milan Mladoniczky, 29.07.2026

What we have for today



Concepts

Why Kubernetes, cluster anatomy, kubectl



Minikube

`minikube start`



App vs Pod

Deploy and access



Replicas and Networking

Scale and publish



Wrap up

Product of our today's workshop

Containers

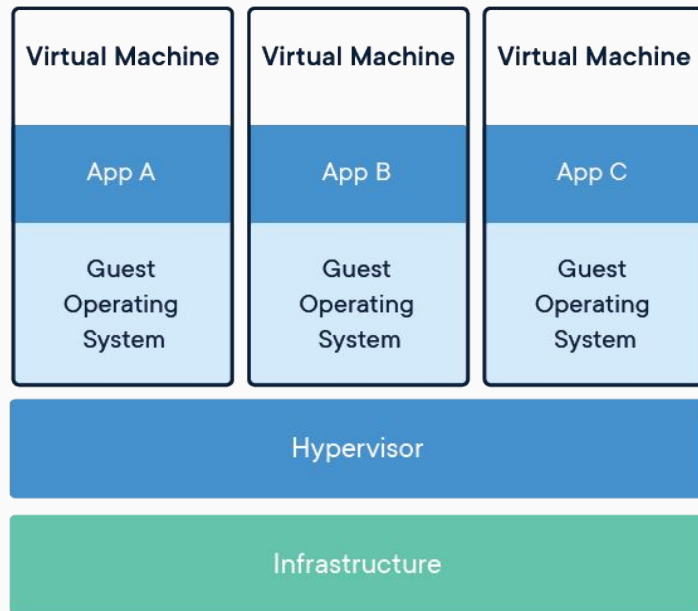
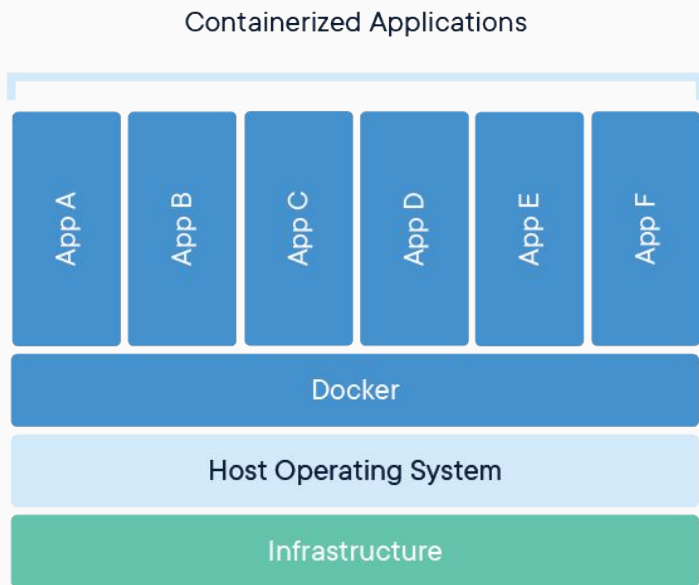


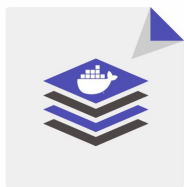
Image to Container

Image is a packaged, read-only template for an application: your code plus everything it needs to run (runtime, libraries, dependencies, config). Think of it as a snapshot or blueprint.

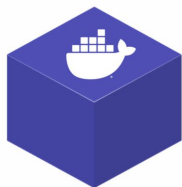
Container is a running instance of an image. When you start a container, you take that read-only image and run it as an isolated process on a machine, with its own file system view, network, and process space, but sharing the host machine's kernel underneath.



Dockerfile



Docker Image



Docker Container

Kubernetes

Kubernetes is an open-source platform for running containerized applications across a cluster of machines.

You describe the state you want, how many copies of an app, how it's exposed, how it should update, and **Kubernetes continuously works to keep the cluster matching** that description.

Google had been running virtually all of its own services in containers for over a decade, using an internal system called Borg.

In 2014, Google released that experience as an open-source project Kubernetes and in 2015 donated it to the newly formed Cloud Native Computing Foundation, which stewards it today.

The name comes from the Greek word for "helmsman" or "pilot", fitting for something that steers containers around a cluster.

Why not just run docker?

Manual container ops

- A host dies - someone SSHes in and restarts containers by hand.
- Traffic spikes - manual docker run on more machines.
- New version - stop, pull, start, hope nothing breaks.
- No consistent record of what's supposed to be running where.

How Kubernetes helps

- **Failover** - dead Pods are recreated automatically.
- **Scaling** - add/remove replicas on command or by policy.
- **Rolling updates** - change versions with zero downtime, and roll back.
- **Scheduling** - the cluster decides which node runs what.

Nodes

Control Plane

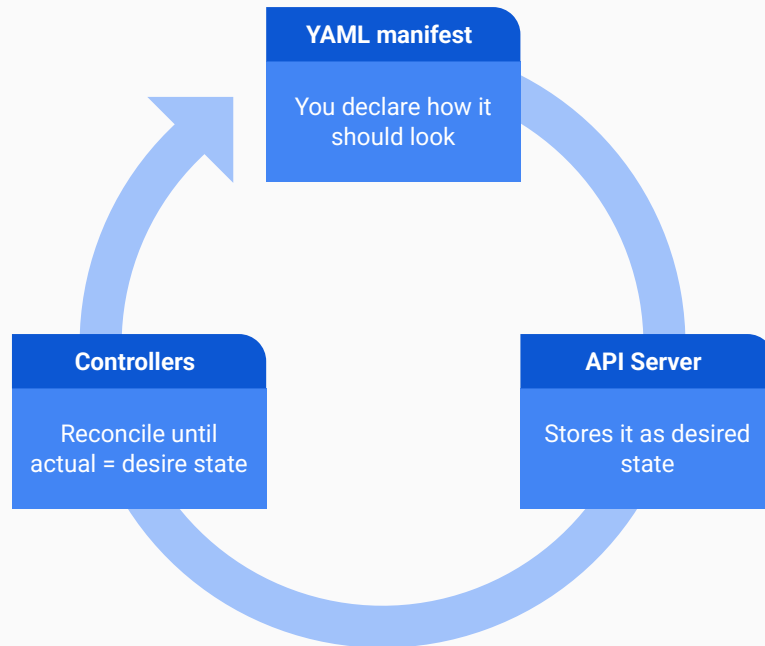
- **API server** - the front door for every request.
- **Scheduler** - decides which node runs a new Pod.
- **Controller** manager - runs reconciliation loops.
- **etcd** - stores the cluster's desired & current state.

Worker

- **kubelet** - talks to the API server, keeps Pods running.
- **kube-proxy** - implements Service networking rules.
- **Container runtime** - actually runs the containers. (docker, containerd)
- Many worker nodes make up the data plane.

In today's workshop, Minikube runs both roles on a single node.

Desired State vs. Actual State - Reconciliation loop



You never tell Kubernetes the steps to take - you describe the end state, and Kubernetes' control loops keep working until reality matches it.

Hierarchy of resources



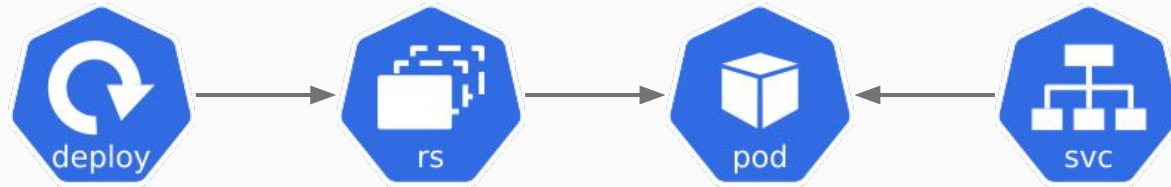
Pod as basic building block, wrapping containers.

Hierarchy of resources



Deployment to better manage replicas and recovery.

Hierarchy of resources



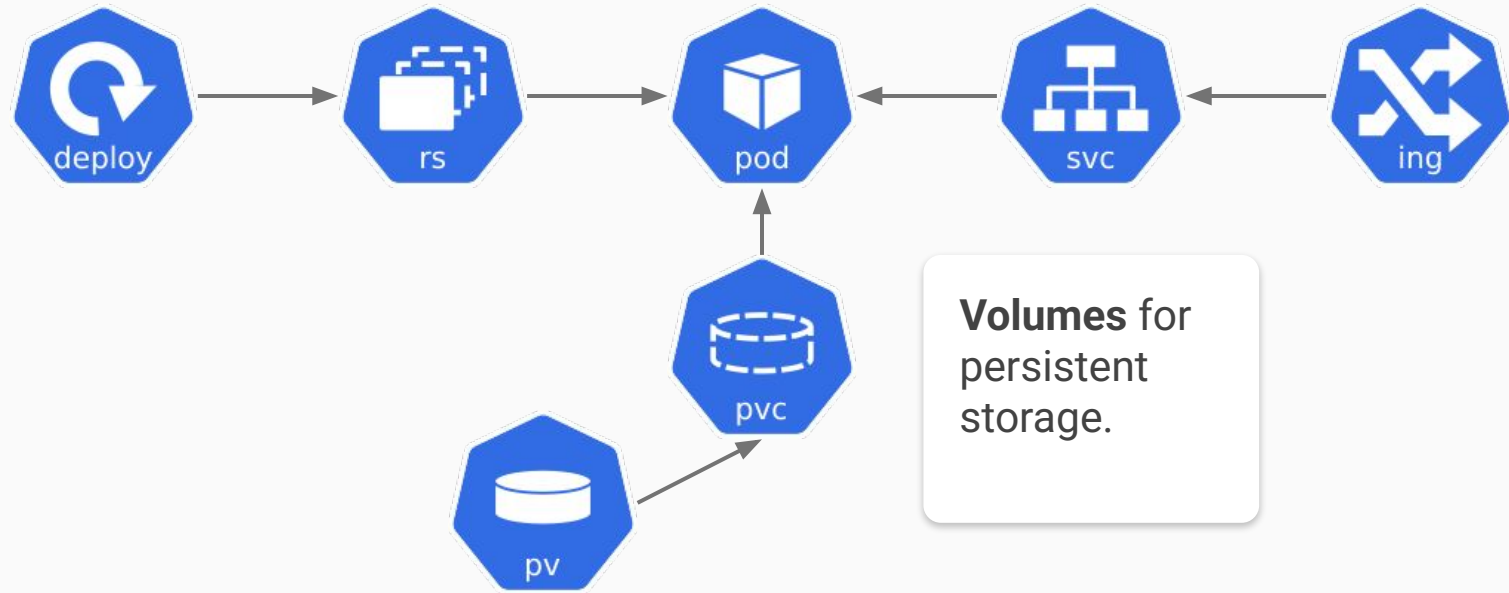
Service gives stable address for group of pods. (i.e. load-balancing)

Hierarchy of resources

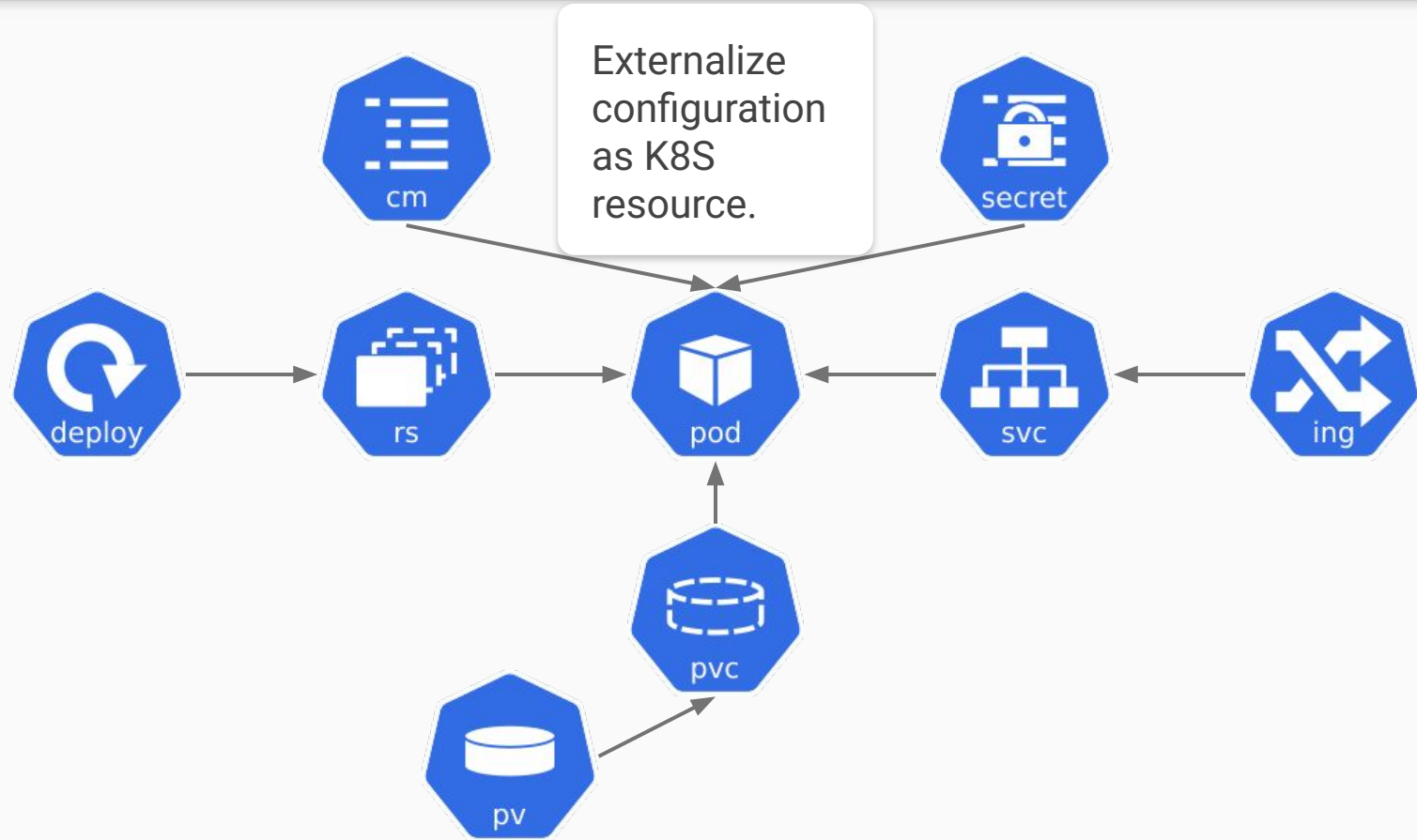


Ingress to
expose
application
outside cluster

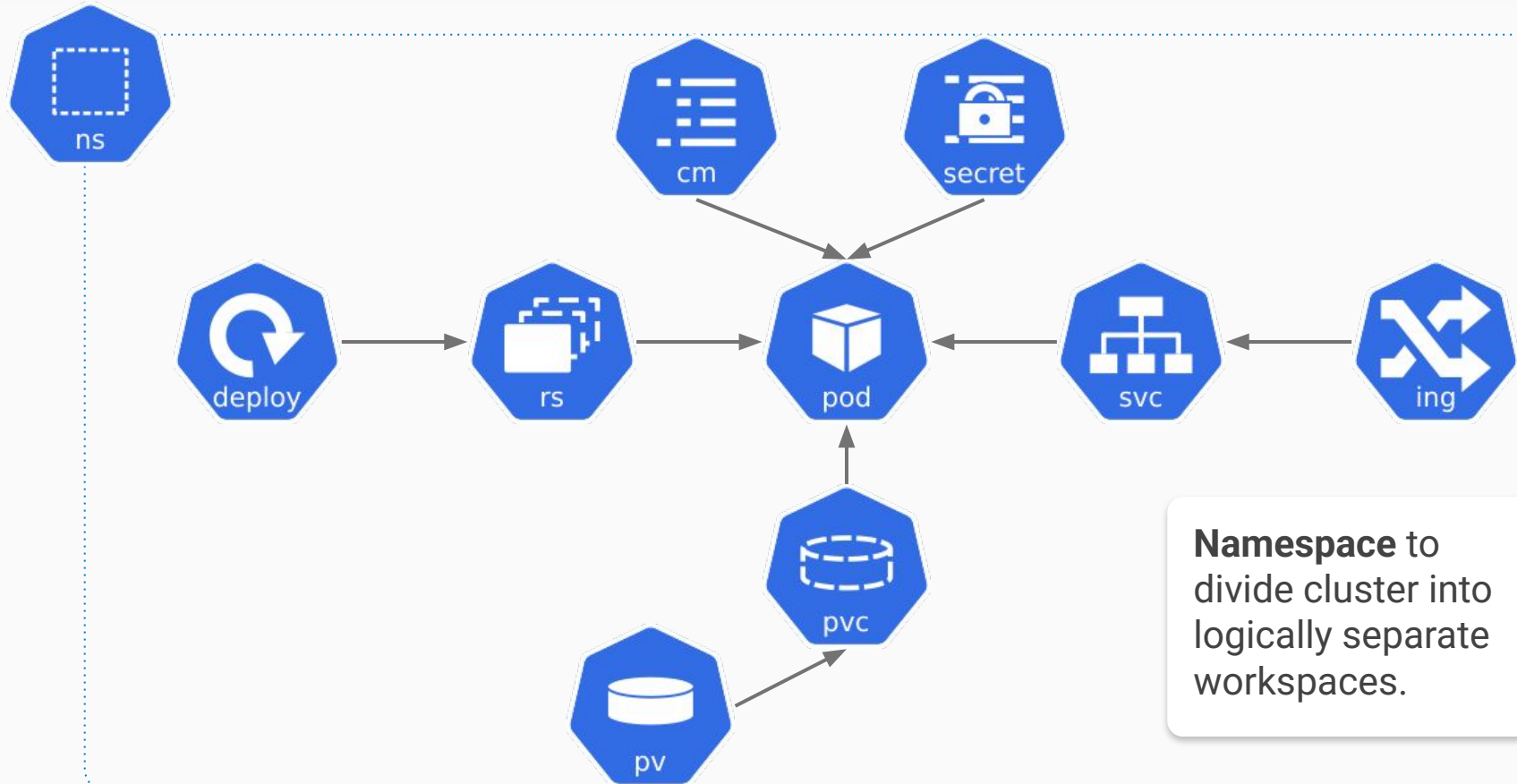
Hierarchy of resources



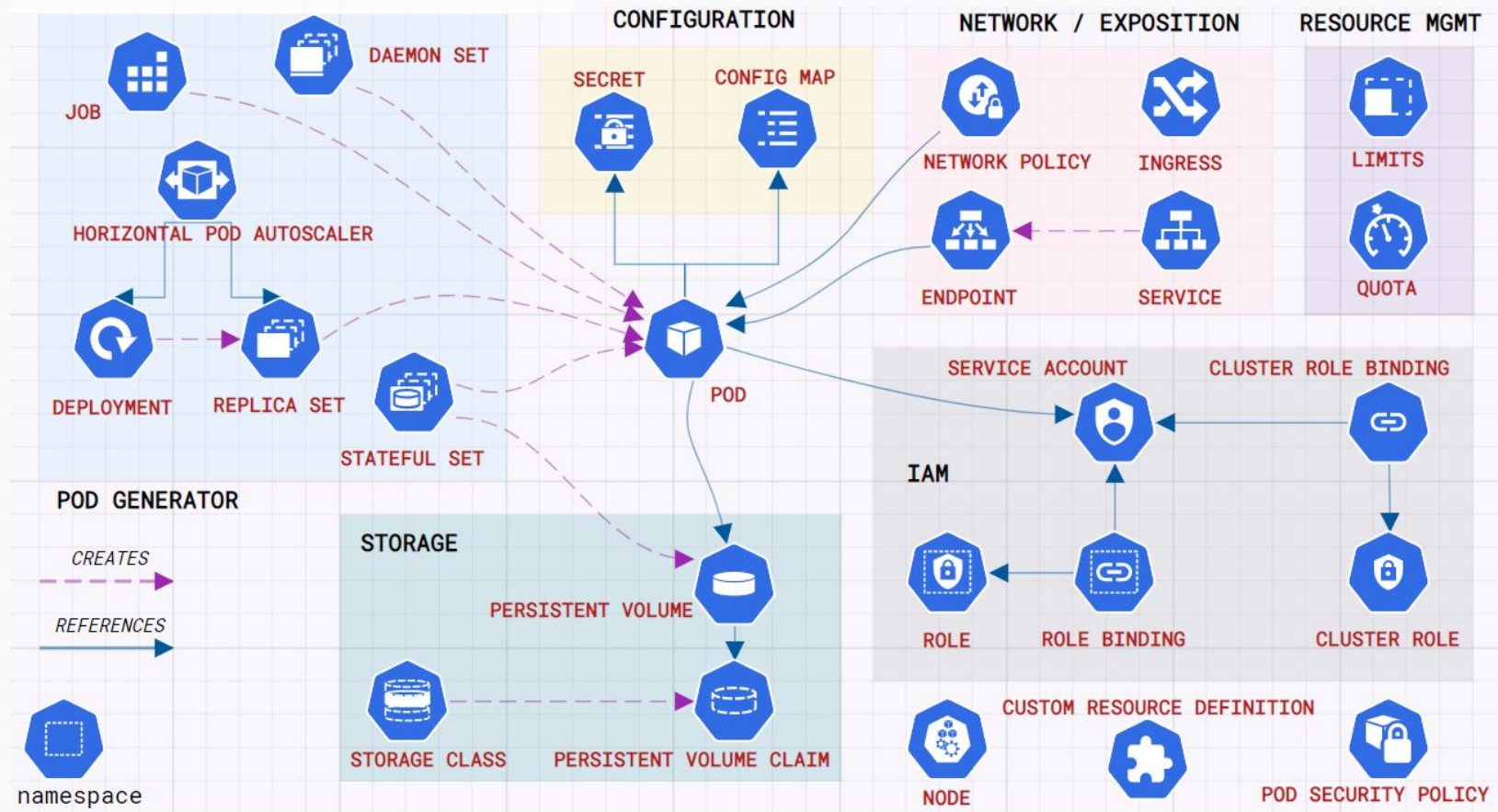
Hierarchy of resources



Hierarchy of resources



and there is a lot more ...



Pod lifecycle

- **Pending** — accepted by the API server, but not yet running. Usually waiting to be scheduled to a node, or waiting on the image to be pulled.
- **Creating** — pulled an image a preparing to start it.
- **Running** — at least one container is running (or starting/restarting). This is not the same as "ready to serve traffic".
- **Succeeded** — all containers exited with code 0 and won't restart. Normal for one-off jobs, not for long-running apps.
- **Failed** — all containers exited, and at least one exited with an error.
- **Unknown** — the node can't be reached, so the control plane can't confirm state.

Pod lifecycle

- **Pending** — accepted by the API server, but not yet running. Usually waiting to be scheduled to a node, or waiting on the image to be pulled.
- **Creating** — pulled an image a preparing to start it.
- **Running** — at least one container is running (or starting/restarting). This is not the same as "ready to serve traffic".
- **Succeeded** — all containers exited with code 0 and won't restart. Normal for one-off jobs, not for long-running apps.
- **Failed** — all containers exited, and at least one exited with an error.
- **Unknown** — the node can't be reached, so the control plane can't confirm state.

READY - separate condition from *Running*. All kubernetes probes are active and pod is ready to receive traffic.

Networking fundamentals

Every Pod get its own IP

Flat pod network -
any Pod can reach
any other Pod
directly.

Pod IP is ephemeral

A recreated Pod get
a new IP.

Container Network Interface

Wires up that pod
network across
every node in the
cluster.

Service to the rescue

Service solves the IP
problem - ClusterIP,
NodePort,
LoadBalancer.

CoreDNS ties it together

Service names
resolve to a stable
virtual IP
automatically.

CoreDNS = cluster local DNS

- DNS Service is system pod (namespace kube-system)
- Every pod is configured automatically with local DNS server
- Every service get stable DNS name

Naming pattern

<service>.<namespace>.svc.cluster.local

Same namespace -> `proto://service-name`

Across namespace -> `proto://service.namespace`

Full FQDN for cross-cluster communication.

Let's start a cluster

Starting minikube

Production Kubernetes runs across many nodes. Minikube shrinks that whole model down to one machine, purely for local development and learning.

Control plane + worker functionality on that one node.

```
$ minikube start
```

```
$ minikube addons enable dashboard
```

```
$ minikube addons enable metrics-server
```

```
$ minikube config set memory 4096
```

```
$ minikube stop
```

```
$ minikube start
```

```
$ minikube ip
```

kubectl

`kubectl` is the command-line tool used to talk to a Kubernetes cluster.

It doesn't run anything itself, every command sends a request to the cluster's API server, which is the front door for the entire cluster.

```
$ kubectl get nodes
```

```
$ kubectl describe node minikube
```

```
$ kubectl get pods -ALL
```

```
$ kubectl get namespaces
```

```
$ kubectl get pods
```

```
$ kubectl cluster-info
```

Quick Pod

Imperative - directly with kubectl or API.

Declarative - manifest file as desired state.

Inspect changes to cluster.

```
$ kubectl run nginx --image=nginx  
pod/nginx created
```

```
$ kubectl apply -f pod.yaml  
pod/nginx configured
```

```
$ kubectl describe pod nginx
```

```
$ kubectl logs nginx
```

```
$ kubectl exec -it nginx -- sh
```


Quick Pod

Imperative - directly with kubectl or API.

Declarative - manifest file as desired state.

Inspect changes to cluster.

```
$ kubectl run nginx --image=nginx  
pod/nginx created
```

```
$ kubectl apply -f pod.yaml  
pod/nginx configured
```

```
$ kubectl describe pod nginx
```

```
$ kubectl logs nginx
```

```
$ kubectl exec -it nginx -- sh
```

```
$ kubectl delete pod nginx  
pod "nginx" deleted from default  
namespace
```

Quick Pod

Imperative - directly with kubectl or API.

Declarative - manifest file as desired state.

Inspect changes to cluster.

```
$ kubectl run nginx --image=nginx
pod/nginx created

$ kubectl apply -f pod.yaml
pod/nginx configured

$ kubectl describe pod nginx

$ kubectl logs nginx

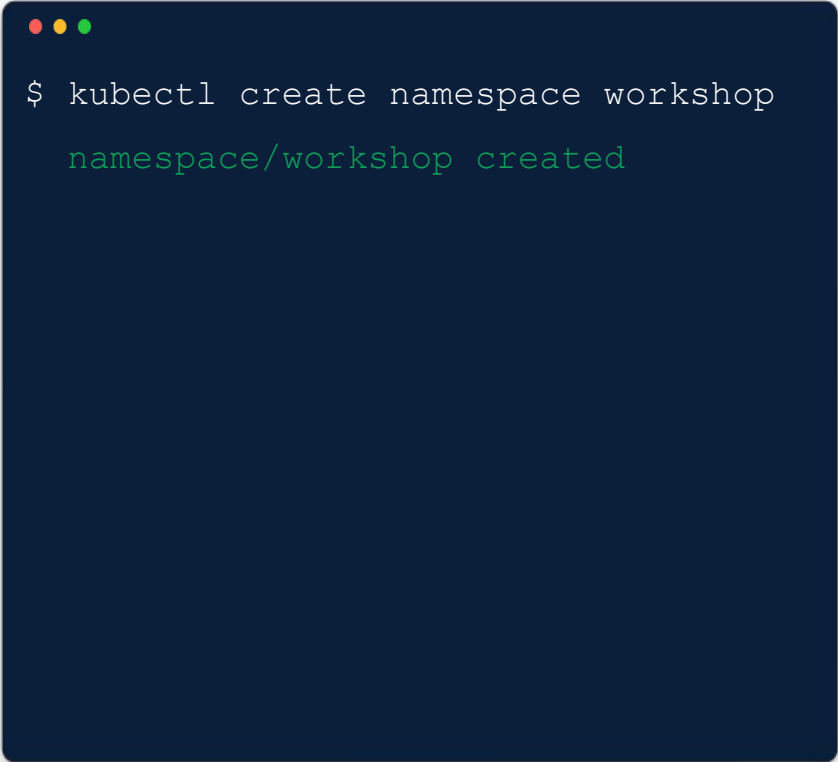
$ kubectl exec -it nginx -- sh

$ kubectl delete pod nginx
pod "nginx" deleted from default
```

No self-healing - Pod is just gone. It is not supervised by any controller.

Deploy a simple app

<https://github.com/netgrif/kubernetes-workshop>



```
$ kubectl create namespace workshop  
namespace/workshop created
```

But first create a
namespace!

podinfo

A deliberately tiny, dependency-free Node.js app for demonstrating how Kubernetes works. It has no `npm install` step at all — just Node's built-in `http` module.

<https://github.com/netgrif/kubernetes-workshop/tree/main/podinfo>

`docker pull netgrif/podinfo`


podinfo

Refresh this page repeatedly (or hammer it with a load test) while scaled to multiple replicas to watch requests bounce between pods.

instance c35efb6e

Pod identity

Pod name	podinfo
Pod IP	10.244.0.26
Pod UID	ae4924b9-0679-4deb-ad22-4e2a28eb6de9
Namespace	default
Node name	minikube
Service account	default
Container hostname (os.hostname())	podinfo
Process instance ID (random, set at startup)	c35efb6e
Log file (see /logs)	pod-c35efb6e.log

Request / client info

Client socket address	::ffff:127.0.0.1
X-Forwarded-For header	(not present - no proxy in front, or not set)
Request method	GET
Request URL	/
User-Agent	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36
Host header	localhost:8080

Runtime / resource info

Server started at	2026-07-27T22:13:16.783Z
Current server time	2026-07-27T22:13:24.308Z
Process uptime	0d 0h 0m 7s
Node.js version	v24.18.0
Platform / Arch	linux / x64
CPU count	2
Total memory	5918 MB
Free memory	3990 MB

deployment

Create - Creating deployment and replica set of size 1 and starting a pod.

Delete pod - Deployment takes care of starting a fresh pod.

Scale - Scaling replica set to more pods.

Update with probes - Activate probes to check for pod state.

```
$ kubectl apply -f deployment.yaml -n workshop
```

```
$ kubectl delete pod -l app=podinfo -n workshop
```

```
$ kubectl scale deployment/podinfo -n workshop --replicas=3
```

```
$ kubectl apply -f deployment-probes.yaml -n workshop
```

services

ClusterIP - for internal user only.

NodePort - reserve own port on the cluster.

LoadBalancer - use provider load-balancer for direct traffic.

```
$ kubectl apply -f  
service-clusterip.yaml -n workshop
```

```
$ kubectl apply -f  
service-nodeport.yaml -n workshop
```

```
$ kubectl apply -f  
service-loadbalancer.yaml -n workshop
```

```
$ kubectl get svc -n workshop
```

```
$ minikube service list
```

How to persist some data

PVC

Check storage class

Create PVC - claim a space for a pod.

Create deployment with mounted pvc - storage has meaning only when is mounted to a pod.

```
$ kubectl get storageclass  
standard
```

```
$ kubectl apply -f pvc.yaml -n workshop
```

```
$ kubectl apply -f deployment.yaml -n  
workshop
```

```
$ minikube sh  
> cd /tmp/hostpath-provisioner  
> cd workshop  
> ls  
pod-<node instance id>.log
```

Externalized Configuration

Configmap - even files could be configs.

Secret - if only container should know something.

```
$ kubectl apply -f configmap.yaml -n workshop
```

```
$ kubectl apply -f secret.yaml -n workshop
```

```
$ kubectl apply -f deployment-config.yaml -n workshop
```

```
$ kubectl exec -it pod/podinfo-<pod id> -n workshop -- sh  
> cd mnt  
> ls  
> cat config/config.json
```

More control for pod order

Stateful Apps

Stable, predictable names — app-0, app-1, app-2 (not random suffixes), which stay the same across restarts.

Stable storage per Pod — each replica gets its own persistent volume that follows it, even if it's rescheduled.

Ordered start-up/scale-down — Pod 0 starts before Pod 1, and shuts down in reverse order.

Accessible via headless service - for direct access to a pod.

```
$ kubectl apply -f statefulset.yaml -n workshop
```

```
$ kubectl apply -f service-headless.yaml -n workshop
```

```
$ kubectl scale sts/podinfo -n workshop --replicas=3
```

```
$ kubectl get pvc -n workshop  
log-storage-podinfo-0  
log-storage-podinfo-1  
log-storage-podinfo-2
```

Publishing the app

Ingress

HTTP-aware gateway into the cluster.

- Host/path-based routing
- One entry point, many Services
 - avoids provisioning a LoadBalancer per app
- TLS termination
- L7 features — things like rewrites, redirects, per-request load balancing (which a plain Service can't do)

```
$ minikube addons enable ingress
$ echo "$(minikube ip) podinfo.local" |
sudo tee -a /etc/hosts
```

```
$ kubectl get ingressClass
nginx
```

```
$ kubectl apply -f
deployment-and-service.yaml -n workshop
```

```
$ kubectl apply -f ingress.yaml -n
workshop
```

What else can be done

- DaemonSets
- RBAC and Monitoring
- Kubernetes distributions
- Plugins
- Kustomize
- Helm

Thank you for
your attention

<https://github.com/netgrif/kubernetes-workshop>

<https://tinyurl.com/3uwwc7ud>